AiDiy Discord 操作編
操作ガイドとセルフテスト

総合評価

✅ OK（記載した確認範囲）。Windowsの実パネルとDiscord専用チャンネルで、起動・自動接続、初回設定、投稿へのAI回答、会話継続、モデル設定の取消と再接続、切断・終了、状態確認等のコマンドを確認しました。音声会話は環境制限のため未テストで、総合判定の対象外です。

1〜4章が操作ガイド、5章が追加テスト、末尾の6章が音声会話です。現行画面20点を掲載しています。Nodeの自動・隔離試験130件、Pythonの30件が成功し、型検査と配布画面の一致も確認しました。PDF版を参照できます。

始める前に

- aidiy_discordとHermesの導入・認証を済ませます。導入とサービス側の設定はDiscord READMEを参照します。
- 専用チャンネルと許可利用者を設定します。Botトークンは画面や資料に載せません。
- Diagの起動と作業フォルダ選択はシステム起動編とランチャー編を参照します。

DiscordはGatewayのWebSocketでイベントを受信します。ポーリングの待ち時間は入りません。音声会話はCoreとLiveAIの設定を使いますが、この冊子の試験ではLiveをOFFにしています。

1. 起動して接続する

1. Diagで作業フォルダを選び、ミニランチャーの「Discord」を押します。端末からはaidiy_discordで起動できます。
2. 「プロジェクト」が選んだフォルダであることを確認します。
3. 初回は自動接続せず、モデル設定画面から始まります。コードはaidiy_hermes固定で、モデルを選んで「接続」を押します。
4. 保存済みの設定がある場合は、起動時に前回の設定で自動接続します。

完了の目印： 左上の「接続済み」と緑の点、右上の「切断」、「CODE ON」、下部の現在のモデル表示です。音声会話の状態は「LIVE ON/OFF」で確認します。

ミニランチャーのDiscordを押して、選んだプロジェクトのパネルを開きます。

図101：ミニ表示



保存設定がない条件では、未接続の設定画面で停止しました。コードはaidiy_hermes、モデルはautoです。接続前に保存ファイルが作られないことも確認しました。LiveはOFFです。

図102：初回起動



保存済みの設定で起動すると自動接続しました。初回画面の「接続」による保存も確認し、選択内容は元の設定と一致しました。

図103：自動接続



2. 専用チャンネルで話しかける

1. 許可した利用者で、設定した専用テキストチャンネルを開きます。
2. 短い文を普通に投稿します。メンションや接頭辞は不要です。
3. 投稿にAIの回答が返ることと、パネル中央に最後のやり取りが表示されることを確認します。
4. 同じ利用者・チャンネルで続けて質問し、前の投稿を踏まえた回答かを読みます。

完了の目印： Discord上で回答を読めることです。パネル表示だけで、投稿先への回答成功とは判定しません。パネルには最後のやり取りを1件表示し、約1分で消します。

確認にはHermesの`auto`を使い、「おはよう。合言葉は『青い星』です。短く挨拶し、合言葉を確認してください。」と投稿しました。続けて「さっきの合言葉を一言で教えてください。」と尋ねています。

投稿を受け付け、パネルに利用者の本文を表示しました。投稿者名も表示します。

図201：投稿表示



AIは「おはようございます。合言葉は『青い星』ですね。」と返しました。空応答やエラーではなく、入力内容に合う回答を確認しています。

図202：回答表示



Discordの専用チャンネル「コード」で、投稿・AI回答・続きの質問を確認できます。合言葉を質問に書き直さなくても「青い星」と返り、会話の文脈が引き継がれました。

図203：チャンネルの回答



3. モデルを選び直す

1. 下部の「モデル ▾」を押します。接続中は、接続を続けたまま設定欄が開きます。
2. コードは`aidiy_hermes`の固定表示です。モデルを一覧から選びます。
3. 「キャンセル」は変更を戻し、接続を続けます。「適用して再接続」は切断・保存・再接続します。
4. 再び「接続済み」になり、下部のモデル表示が選択内容と一致することを確認します。

設定欄のボタンが見えない場合は、欄内を下へスクロールします。`auto`は自動判定、`same`は変更なし、`codex_cli/same`はCLI指定、`openai_oauth/…`はHermesのプロバイダとモデルの指定です。選択は`~/.aidiy/aidiy_discord.json`に保存します。

接続済みのまま「モデル」を開きました。固定のコード欄と、編集できるモデル欄が表示されます。

図301：接続中のモデル設定



モデルをsameへ変更したところです。「キャンセル」と「適用して再接続」を区別して操作します。

図302：モデル選択例



キャンセルするとautoの表示へ戻り、接続は継続しました。保存ファイルもautoのままでした。

図303：取消後の接続継続



設定欄を開き直し、元のautoで「適用して再接続」を押します。

図304：適用して再接続



再び接続済みとなり、モデルはaidiy_hermes - autoです。保存値の一致も確認しました。

図305：再接続完了



4. 切断・再接続・終了する

1. 右上の「切断」を押します。「未接続」のモデル設定画面に戻ります。
2. 再び使うときは設定を確認し、「接続」を押します。
3. 終了するときは右上の「×」を押します。接続中は終了確認が表示されます。
4. 「キャンセル」なら接続を続け、「終了」ならパネル・Bot・子プロセスを終了します。切断中は確認なしで終了します。

完了の目印： 切断は「未接続」と「接続」、再接続は「接続済み」、終了はパネルが閉じたことを確認します。停止の確認では、このアプリのBotや子プロセスも照合します。

「切断」で未接続の設定画面へ戻りました。

図401：切断



「接続」で再び接続済みとなりました。

図402：再接続



接続中に「×」を押すと、終了確認を表示します。

図403：接続中の終了確認



キャンセルすると確認が閉じ、接続を継続しました。確認を開き直した「終了」ではパネルが閉じました。切断中の「×」は確認なしで終了することも確認済みです。

図404：終了取消で接続継続



5. 追加テスト

状態確認と会話の操作

専用チャンネルへ`!aidiy status`を送り、「Code: 待機中 / Live: OFF」が返りました。

図501：状態コマンド



!aidiy helpを送り、専用チャンネルへの話しかけ方が返りました。

図502：ヘルプ表示



処理がない状態の`!aidiy stop`には「実行中のコード要求はありません。」が返りました。実行中の要求を中断した確認とは区別します。

図503：待機中の停止



!aidiy newに、新しいコード会話を開始する案内が返りました。

図504：会話のリセット



単一起動

接続中にaidiy_discordをもう一度起動しました。同じパネルが表示され、ウィンドウIDとパネル1枚の維持を照合しました。

図505：単一起動



確認した条件

対象	確認結果
初回設定・保存・自動接続	実画面で確認
AI回答・会話継続・コマンド	実チャンネルで確認
モデル取消・適用と再接続	実画面と保存値で確認
接続中の終了確認・取消、切断中の終了	実画面で確認

通信方式と表示

DiscordはGatewayのWebSocketでイベントを受信します。Slackの通常モード／アイドリングモードに相当するポーリングは行いません。AIの回答生成や外部通信の時間は別にかかります。

自動・隔離試験と限界

Nodeの130件、Pythonの30件が成功しました。型検査と配布画面の一致も確認しています。自動試験は合成通信や一時設定を含み、実サービスの確認は各章の投稿・AI回答・画面結果で判定します。

VPN・社内プロキシ経由の実接続、Mac／Linux実機、長時間連続運転は未実施・判定対象外です。実行中の長いコード要求の中断は、実チャンネルでは未実施です。

終了時の状態

モデル設定は元の内容へ復元し、試験用パネルとBotを終了しました。Diagと共有サービスは継続稼働しています。試験投稿は専用チャンネルに残しています。

6. 音声会話

環境制限のため、未テストです。 ボイスチャンネルへの入室、双方向音声、退出による切断、モニター再生は、総合判定の対象外です。

テキストの試験ではLiveをOFFにしています。音声WebSocket等の自動・隔離試験の成功を、実際の音声会話の成功とは扱いません。未テストの音声画面にエビデンスや合格判定は付けません。